

**CO3-AT1-Assignment**

View Only

← Back

**QUESTIONS**
**Q1**

Banking Transaction System –  
Exception Handling  
10 marks

**Q2**

Hospital Patient Registration –  
User-Defined Exception  
10 marks

**Q3**

E-Commerce Order Processing  
– Built-in Exceptions  
10 marks

**Q4**

Online Food Delivery – Multiple  
Threads and Execution Order  
10 marks

**Q5**

Manufacturing Plant –  
Producer-Consumer Problem  
10 marks

5/5 answered

**Q1 Banking Transaction System – Exception Handling**

10 marks

Develop a Java program for an ATM withdrawal system. The program accepts the account balance and withdrawal amount. The program must:

- Accept the current account balance and withdrawal amount.
- Throw a user-defined `InsufficientBalanceException` when the withdrawal amount exceeds the balance.
- Handle withdrawal amounts that are zero or negative.
- Handle invalid numeric input using a suitable built-in exception.
- Display a meaningful transaction result.
- Use try-catch-finally so that the transaction completion message is always displayed.

Sample Test Cases:

Input / Scenario Expected Result Purpose

10000 3000 Normal case

10000 12000 Boundary / exception case

10000 -500 Invalid input

abc 3000 Invalid input

Your Code

```
1 import java.util.*;
2
3 class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         try {
8             double balance = sc.nextDouble();
```

Input

10000 3000

Output

Withdrawal successful

```
7      try {
8          double balance = sc.nextDouble();
9          double amount = sc.nextDouble();
10
11         if (amount <= 0)
12             throw new IllegalArgumentException();
13
14         if (amount > balance)
15             throw new InsufficientBalanceException();
16
17         balance -= amount;
18         System.out.println("Withdrawal successful");
19         System.out.println("Remaining balance: " + balance);
20
21     } catch (InsufficientBalanceException e) {
22         System.out.println("Insufficient balance");
23     } catch (InputMismatchException e) {
24         System.out.println("Invalid input");
25     } catch (IllegalArgumentException e) {
26         System.out.println("Invalid withdrawal amount");
27     } finally {
28         System.out.println("Transaction completed");
29     }
30 }
31
32
33 class InsufficientBalanceException extends Exception {
34 }
```

Output

```
Withdrawal successful
Remaining balance: 7000.0
Transaction completed
```

 Submitted

## QUESTIONS

Q1

Banking Transaction System –  
Exception Handling  
10 marks

Q2

Hospital Patient Registration –  
User-Defined Exception  
10 marks

Q3

E-Commerce Order Processing  
– Built-in Exceptions  
10 marks

Q4

Online Food Delivery – Multiple  
Threads and Execution Order  
10 marks

Q5

Manufacturing Plant –  
Producer-Consumer Problem  
10 marks

5/5 answered

## Q2 Hospital Patient Registration – User-Defined Exception

10 marks

Develop a Java program for a hospital patient registration system. The program accepts the patient's name, age, and body temperature. Create suitable user-defined exception classes for invalid age and invalid temperature. Validate the input before registration. Use try-catch-finally and display a registration completion message from finally.

Sample Test Cases:

Input / Scenario Expected Result Purpose

Arun, 35, 98.6 Patient registered successfully Normal case

Meena, 12, 98.4 InvalidAgeException Boundary / exception case

Ravi, 45, 105.2 InvalidTemperatureException Invalid input

Kumar, -2, 98.1 InvalidAgeException Invalid input

## Your Code

```
1 import java.util.*;  
2  
3 class Main {  
4  
5     static class InvalidAgeException extends Exception {  
6     }  
7  
8     static class InvalidTemperatureException extends Exception {  
9     }  
10  
11     public static void main(String[] args) {  
12         Scanner sc = new Scanner(System.in);  
13  
14         try {  
15             String name = sc.next();  
16             int age = sc.nextInt();  
17             double temp = sc.nextDouble();
```

## Input

Arun 35 98.6

## Output

```
Patient registered successfully  
Registration completed
```

5/5 answered

```
8 static class InvalidTemperatureException extends Exception {  
9     }  
10  
11 public static void main(String[] args) {  
12     Scanner sc = new Scanner(System.in);  
13  
14     try {  
15         String name = sc.next();  
16         int age = sc.nextInt();  
17         double temp = sc.nextDouble();  
18  
19         if (age < 18 || age > 120)  
20             throw new InvalidAgeException();  
21  
22         if (temp < 95 || temp > 100)  
23             throw new InvalidTemperatureException();  
24  
25         System.out.println("Patient registered successfully");  
26  
27     } catch (InvalidAgeException e) {  
28         System.out.println("InvalidAgeException");  
29     } catch (InvalidTemperatureException e) {  
30         System.out.println("InvalidTemperatureException");  
31     } finally {  
32         System.out.println("Registration completed");  
33     }  
34 }  
35 }
```

Registration completed

 Submitted

## CO3-AT1-Assignment

View Only

← Back

## QUESTIONS

Q1

Banking Transaction System –  
Exception Handling

10 marks

Q2

Hospital Patient Registration –  
User-Defined Exception

10 marks

Q3

E-Commerce Order Processing  
– Built-in Exceptions

10 marks

Q4

Online Food Delivery – Multiple  
Threads and Execution Order

10 marks

Q5

Manufacturing Plant –  
Producer-Consumer Problem

10 marks

5/5 answered

## Q3 E-Commerce Order Processing – Built-in Exceptions

10 marks

Develop a Java program for an online shopping application. Store product names and prices in arrays and accept a product index and quantity from the customer.

The program must:

- Calculate total cost as quantity × price.
- Handle invalid product indices.
- Handle invalid numeric input.
- Reject zero or negative quantities.
- Use appropriate built-in exception classes.
- Continue execution after handling an invalid transaction.

Sample Test Cases:

Input / Scenario Expected Result Purpose

1, 2 Valid product; total cost calculated Normal case

5, 2 `ArrayIndexOutOfBoundsException` handled Boundary / exception case

2, -1 Invalid quantity Invalid input

abc, 2 `InputMismatchException` handled Invalid input

Your Code

```
1 import java.util.*;  
2  
3 class Main {  
4     public static void main(String[] args) {  
5         Scanner sc = new Scanner(System.in);  
6         String[] products={"Laptop", "Phone", "Tablet"};  
7         double[] prices = {150000, 20000, 15000};
```

Input

1 2

Output

Product-Consumer Problem

10 marks

5/5 answered

Your Code

```
1 import java.util.*;
2
3 class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6         String[] products={"Laptop","Phone","Tablet"};
7         double[] prices = {50000,20000,15000};
8         try{
9             int index = sc.nextInt();
10            int quantity = sc.nextInt();
11            if (quantity <= 0)
12                throw new IllegalArgumentException("Invalid quantity");
13            System.out.println("Product: " + products[index]);
14            System.out.println("Total Cost: " + (prices[index] * quantity));
15
16        } catch (ArrayIndexOutOfBoundsException e) {
17            System.out.println("Invalid product index");
18        } catch (InputMismatchException e) {
19            System.out.println("Invalid input");
20        } catch (IllegalArgumentException e) {
21            System.out.println("Invalid quantity");
22        }
23        System.out.println("Transaction completed");
24    }
25 }
```

 Submitted

Input

1 2

Output

```
Product: Phone
Total Cost: 40000.0
Transaction completed
```

## CO3-AT1-Assignment

View Only

← Back

## QUESTIONS

Q1

Banking Transaction System –  
Exception Handling

10 marks

Q2

Hospital Patient Registration –  
User-Defined Exception

10 marks

Q3

E-Commerce Order Processing  
– Built-in Exceptions

10 marks

Q4

Online Food Delivery – Multiple  
Threads and Execution Order

10 marks

Q5

Manufacturing Plant –  
Producer-Consumer Problem

10 marks

5/5 answered

## Q4 Online Food Delivery – Multiple Threads and Execution Order

10 marks

An online food-delivery application uses three threads: Order Processing, Payment Processing, and Delivery Assignment.

Develop a Java program in which these activities are represented by separate threads. Analyze the problem if all three threads are started independently.

Modify the program so that:

- Order processing completes before payment processing.
- Payment processing completes before delivery assignment.
- The program uses appropriate Java thread-management techniques.
- The output clearly shows the required execution order.

Explain the difference between simply starting all threads and explicitly controlling their execution order.

Sample Test Cases:

Input / Scenario Expected Result Analysis Focus

Order=Pizza; Payment=UPI; Delivery=Agent A Order → Payment → Delivery Normal execution

Order=Burger; Payment=Card; Delivery=Agent B Order → Payment → Delivery Concurrency / design

Payment=Failed Delivery assignment should not proceed Exception / boundary

Your Code

```
1 import java.util.*;  
2  
3 class main{  
4     public static void main(String[] args) throws Exception{  
5         Scanner sc = new Scanner(System.in);  
6
```

Input

Pizza  
UPI  
Agent A

Output



5/5 answered

```
1 import java.util.*;
2
3 class main{
4     public static void main(String[] args) throws Exception{
5         Scanner sc = new Scanner(System.in);
6
7         String order = sc.nextLine();
8         String payment = sc.nextLine();
9         String delivery = sc.nextLine();
10
11         Thread t1 = new Thread(() -> System.out.println("Order processing"));
12         Thread t2 = new Thread(() -> System.out.println("Payment processing"));
13         Thread t3 = new Thread(() -> System.out.println("Delivery Assignment"));
14
15         t1.start();
16         t1.join();
17
18         t2.start();
19         t2.join();
20
21         if(!payment.equalsIgnoreCase("Failed")){
22             t3.start();
23             t3.join();
24         }
25     }
26 }
```

Submitted

Pizza  
UPI  
Agent A

Output

```
Order processing
Payment processing
Delivery Assignment
```

## CO3-AT1-Assignment

View Only

← Back

### QUESTIONS

Q1

Banking Transaction System –  
Exception Handling  
10 marks

Q2

Hospital Patient Registration –  
User-Defined Exception  
10 marks

Q3

E-Commerce Order Processing –  
Built-in Exceptions  
10 marks

Q4

Online Food Delivery – Multiple  
Threads and Execution Order  
10 marks

Q5

Manufacturing Plant –  
Producer-Consumer Problem  
10 marks

5/5 answered

### Q5 Manufacturing Plant – Producer-Consumer Problem

10 marks

A manufacturing plant has one production thread and one packaging thread. A shared buffer can contain only one product.

Implement the producer-consumer solution using synchronized, wait(), and notify().

The program must:

- Make the producer wait when the buffer is full.
- Make the consumer wait when the buffer is empty.
- Prevent duplicate consumption.
- Ensure that products are consumed only after production.
- Demonstrate correct coordination between the two threads.

Analyze what incorrect behavior could occur if synchronization and wait/notify are removed.

Sample Test Cases:

Input / Scenario Expected Result Analysis Focus

Products = P1, P2, P3 P1 → P2 → P3 are produced and consumed once each Normal execution

Consumer starts first Consumer waits until a product is available Concurrency / design

Buffer capacity = 1; Products = P1, P2 Producer waits while P1 remains unconsumed Exception / boundary

Your Code

```
1 class Main {  
2     static String product = null;  
3  
4     static synchronized void produce(String p) throws Exception {  
5         while (product != null)  
6             Main.class.wait();  
7     }
```

Input

stdin input

Output



```
14     while (product == null)
15         Main.class.wait();
16
17     System.out.println("Consumed " + product);
18     product = null;
19     Main.class.notify();
20 }
21
22 public static void main(String[] args) throws Exception {
23
24     Thread producer = new Thread(() -> {
25         try {
26             produce("P1");
27             produce("P2");
28             produce("P3");
29         } catch (Exception e) {}
30     });
31
32     Thread consumer = new Thread(() -> {
33         try {
34             consume();
35             consume();
36             consume();
37         } catch (Exception e) {}
38     });
39
40     producer.start();
41     consumer.start();
42
43     producer.join();
44     consumer.join();
45 }
46 }
```